Sprawozdanie Lab 05 (Funkcje,Rekurencje)

1. Cel ćwiczenia:
   - Celem ćwiczenia było:

     Zaimplementowanie funkcji w wersji iteracyjnej i rekurencyjnej dla:
     - ✓ Silni n!
     - ✓ Sumy cyfr liczby
     - ✓ Ciągu Fibonacciego(iteracyjnie i rekurencyjnie)
   - Przeprowadzenie pomiarów czasu działania implementacji iteracyjnych i rekurencyjnych z użyciem biblioteki ***chrono***
   - Ocena wpływu sposobu implementacji(iteracja vs rekurencja) na:
     - Czas działania
     - Zużycie pamięci (stos)
     - Praktyczne ograniczenia (przepełnienie typu, ryzyko przepełnienia stosu)
2. Środowisko testowe:
   - Systemy operacyjne: CachyOS
   - Kompilator i wersja: g++
   - Tryb kompilacji: zwykła
   - Sprzęt(CPU/RAM): `AMD Ryzen 7 9800X3D (16) @ 5.27 GHz / 32GB DDR5`

3. Opis implementacji:
   - silnia_iter(n) – silnia obliczana pętlą ***for***.
   - silnia_rek(n) – silnia obliczana rekurencyjnie z warunkiem bazowym dla ***n<=1***.
   - suma_iter(n) – suma cyfr liczby obliczana w pętli while przez ***n%10***.
   - suma_rek(n) – suma cyfr rekurencyjnie: ostatnia cyfra n%10 + wynik dla ***n/10***.
   - fibon_iter(n) – Fibonacci iteracyjnie w czasie liniowym ***O(n)***.
   - fibon_rek(n) – Fibonacci rekurencyjnie w czasie wykładniczym ***O(2^n)*** (bez memoizacji).

   Zastosowane typy:

   - Silnia i Fibonacci: usigned long long
   - Suma cyfr: unsigned int

4. Metodyka pomiarów czasu:

Do pomiaru czasu wykorzystano:

- std::chrono::high_resolution_clock::now() jako znacznik start/stop,
- wynik w mikrosekundach: duration_cast<microseconds>(end - start).count().

Aby ograniczyć wpływ zakłóceń systemowych:

- dla każdego n wykonywano wiele wywołań funkcji
- pomiary iteracji i rekurencji wykonywano w tym samym programie i dla tych samych danych,
- w sekcji mierzonej nie wykonywano operacji wejścia/wyjścia (I/O), ponieważ zaburzają wyniki.

Zakres testów dobrano w grupach aby obserwować zachowanie dla:

- małych wartości: n = 1..5,
- średnich wartościach: n ~ 8..12
- większych wartościach: n ~ 15..20
  dla fibon_rek(n) – zastosowano ograniczenie powtórzeń dla większych n , ponieważ rekurencyjna implementacja bez optymalizacji staje się bardzo wolna

5. Wyniki pomiarów:

Wyniki zostały wypisane w formie tabeli (kolumny: n, czas iter, czas rek) dla:

- Silni,
- Sumy cyfr,
- Fibonacciego

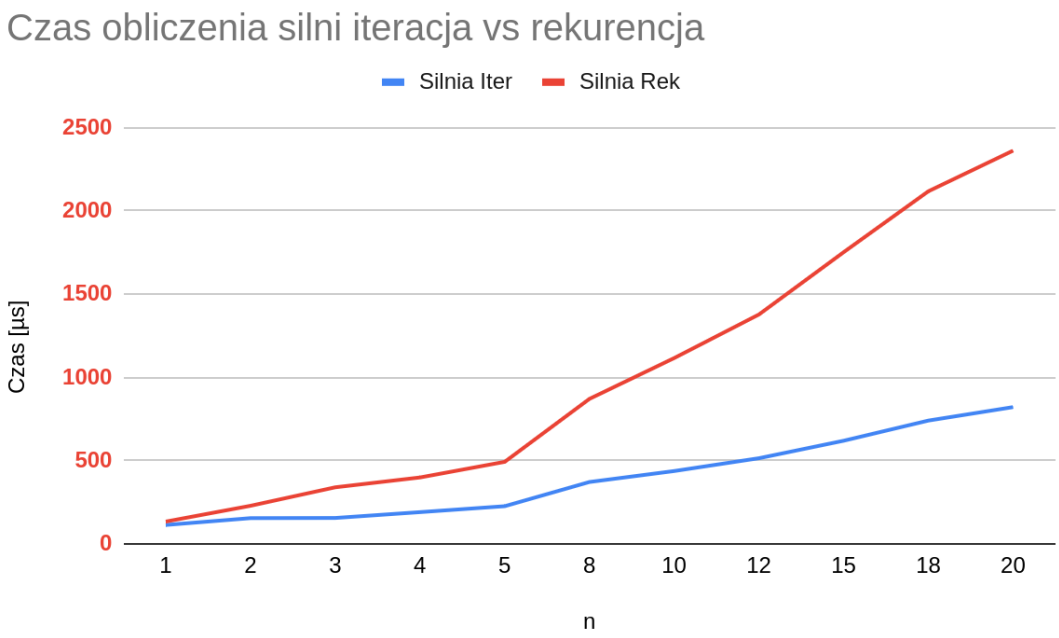
n	Silnia Iter	Silnia Rek	Suma Iter	Suma Rek	Fibon Iter	Fibon Rek
1	117 us	121 us	119 us	230 us	134 us	134 us
2	116 us	220 us	119 us	310 us	190 us	325 us
3	156 us	287 us	121 us	230 us	209 us	517 us
4	195 us	383 us	119 us	229 us	285 us	926 us
5	232 us	459 us	119 us	229 us	339 us	1581 us
8	365 us	688 us	119 us	229 us	502 us	6835 us
10	434 us	867 us	191 us	331 us	703 us	19128 us
12	511 us	1012 us	191 us	299 us	764 us	52353 us
15	639 us	1311 us	195 us	326 us	917 us	204438 us
18	741 us	2138 us	191 us	325 us	1055 us	859139 us
20	805 us	2094 us	191 us	325 us	1148 us	2250831 us

6. Wykres na podstawie danych:

Dane:

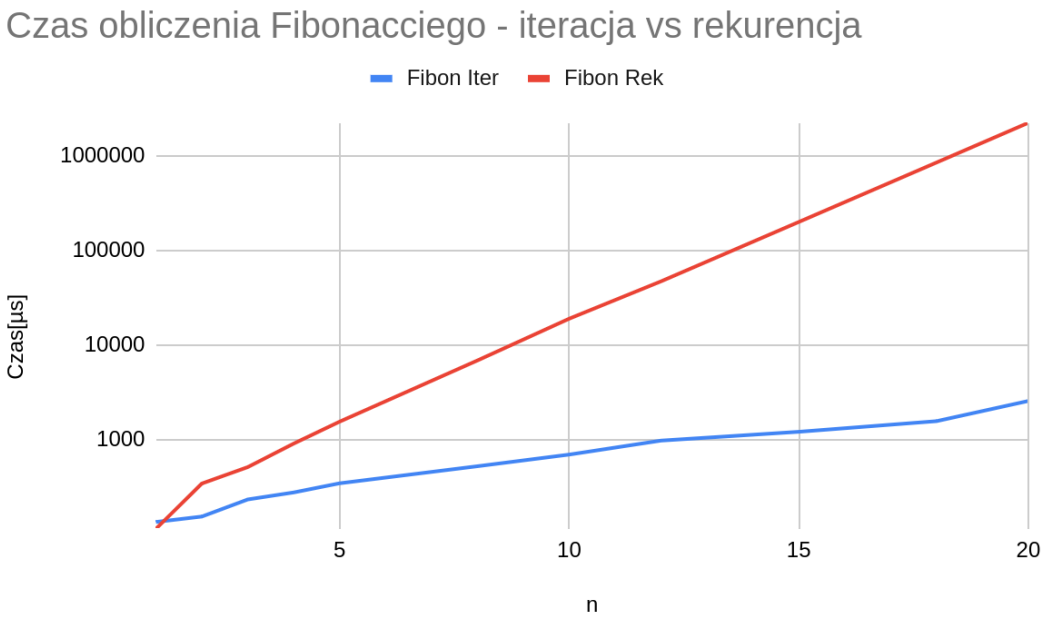
```
❯ cd "/mnt/shared/Programming/cpp/Paradygmaty_programowania/Zadania/Lab05/" && g++ plik.cpp -o /tmp/plik && /tmp/plik
n     Silnia Iter     Silnia Rek      Suma Iter       Suma Rek        Fibon Iter      Fibon Rek
-----------------------------------------------------------------------------------------------------
1     114 us          134 us          137 us          248 us          134 us          113 us
2     155 us          229 us          155 us          269 us          153 us          341 us
3     156 us          340 us          153 us          248 us          232 us          509 us
4     191 us          399 us          134 us          249 us          275 us          902 us
5     227 us          493 us          153 us          248 us          344 us          1539 us
8     372 us          871 us          154 us          248 us          521 us          6836 us
10    438 us          1116 us         210 us          359 us          692 us          18923 us
12    515 us          1378 us         210 us          371 us          973 us          47039 us
15    620 us          1752 us         210 us          363 us          1207 us         199746 us
18    741 us          2118 us         210 us          364 us          1564 us         850695 us
20    822 us          2362 us         210 us          362 us          2557 us         2238828 us
```

Wykres Silnia:



Wykres Fibonacci:

7.
8.



9. Analiza i odpowiedzi na pytania z instrukcji:
   - Czy Iteracja i rekurencja licza takie same maksymalne wartosci?
     Dla poprawnie wykonanych funkcji wyniki beda takie same o ile :
     -nie wystapi przepelnienie typu
     -rekurencja nie doprowadzi do problemow ze stosem

   - Czy czas dzialnia funkcji rekurencyjnych i iteracyjnych jest taki sam:
     -Iteracja jest szybsza, bo nie ma narzuconych wywolan funkcji i nie uzywa stosu do przechowywania kolejnych wywolan
   - -Rekurencja ma dodatkowy koszt wywolan i zyzywa stos, co może pogarszac czas i ograniczac maskymalna glebokosc
     Im większe n tym większe są roznice
   - Jak zmienia się czas dla malych i duzych wartosci?
     * silnia – obie wersje rosna czasowo w przyblizeniu liniowo wzgledem n, rekurencja jest wolniejsza przez narzut wywolan
     * suma cyfr – czas zalezy od liczby cyfr, a nie od wartosci n jako liczby samej w sobie.
   - *Fibonacci – roznica jest najwieksza:
     *iteracja O(n) jest szybka
   - *rekurencja bez optymalizacji O(2^n) robi się bardzo wolna szybko

Wplyw na pamiec (stos):

*w wersji iteracyjnej pamiec jest stala

*w wersji rekurencyjnej kazdy poziom wywolania odklada ramke na stosie , pamiec rosnie wraz z glebokoscia rekurencji. To może prowadzic do bledow przy bardzo duzych n (stack oveflow).

8.Wnioski koncowe:

1. Implementacje iteracyjne i rekurencyjne dają te same wyniki tylko do momentu, gdy nie wystąpi przepełnienie typu oraz gdy rekurencja nie przekroczy ograniczeń stosu.
2. Iteracja jest zwykle szybsza od rekurencji ze względu na brak kosztu wielokrotnych wywołań funkcji i brak narzutu na stos.
3. Dla silni (unsigned long long) maksymalna poprawna wartość to n = 20.
4. Dla Fibonacciego (unsigned long long) maksymalna poprawna wartość to n = 93 (przy F(0)=0, F(1)=1).
5. Rekurencyjny Fibonacci bez memoizacji jest niepraktyczny dla większych n, ponieważ czas rośnie wykładniczo.